Centralized CI CD Platform Using Jenkins

1. Introduction

In modern software development, continuous integration and continuous deployment play a critical role in delivering applications quickly and reliably. Many organizations face challenges due to multiple Jenkins servers, inconsistent pipelines, and lack of standardization.

This project focuses on building a centralized Jenkins platform that allows multiple applications to use standardized CI CD pipelines using a shared library approach.

2. Objective

The main objectives of this project are:

- To set up a centralized Jenkins server on AWS EC2
- To create automated CI CD pipelines
- To integrate Jenkins with GitHub
- To implement shared pipeline libraries
- To support multiple applications using a single pipeline structure

3. Tools and Technologies Used

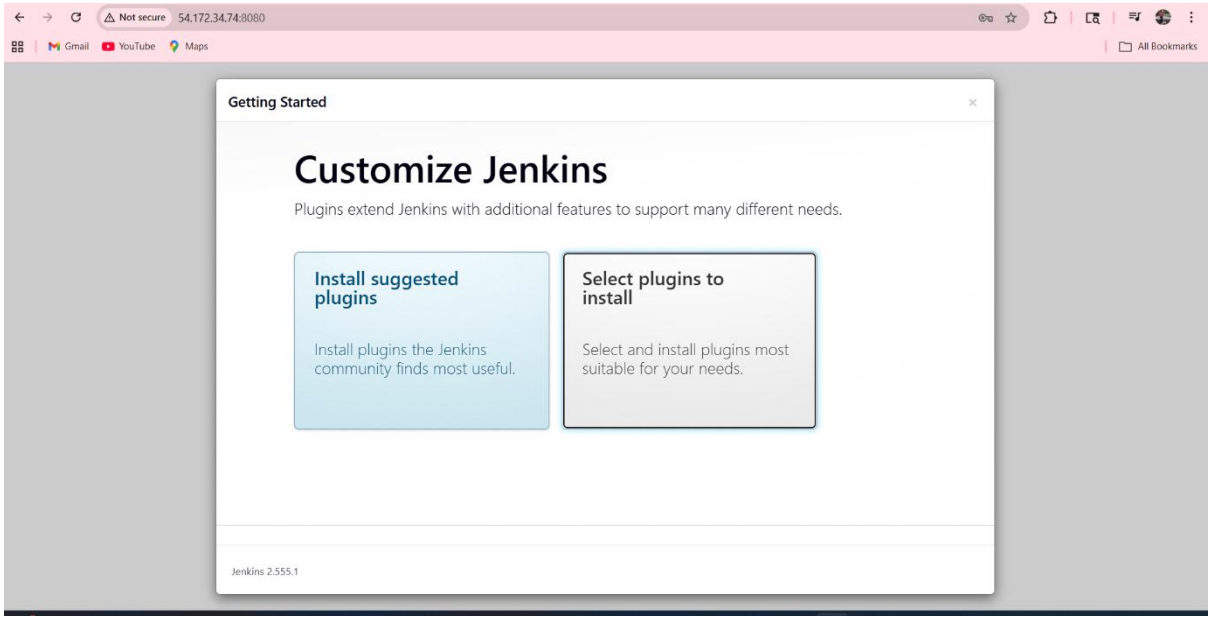
- Jenkins
- Amazon EC2
- GitHub
- Git
- Docker
- Linux Ubuntu

4. Jenkins Installation and Setup

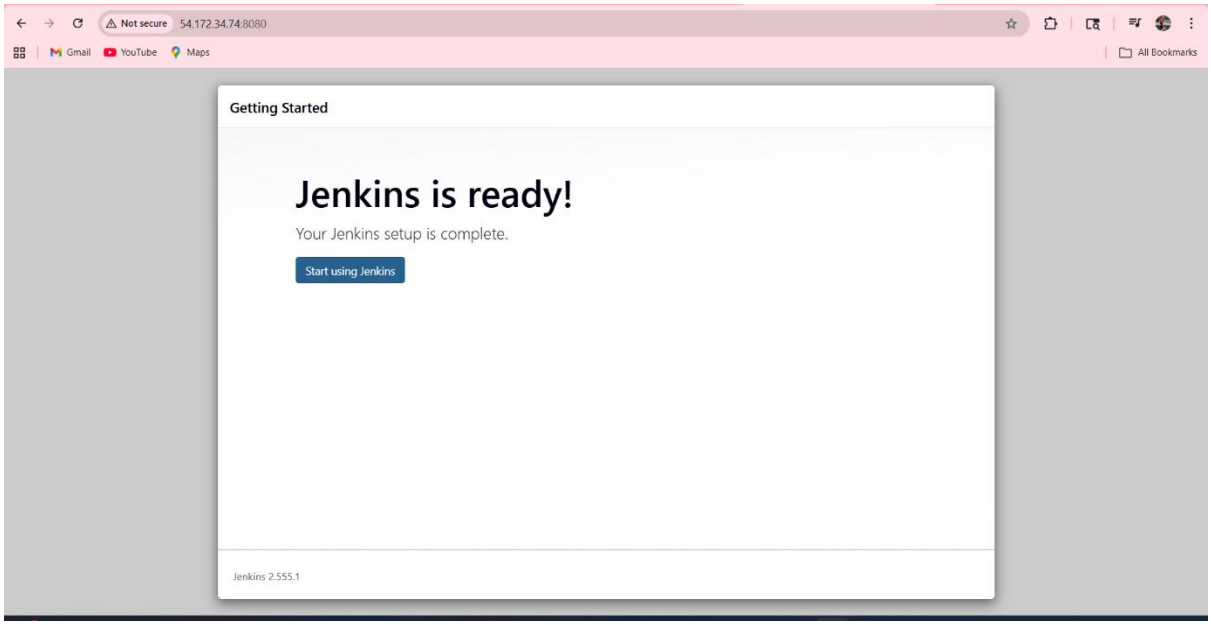
Jenkins was installed on an EC2 instance using Ubuntu operating system. Java was installed as a prerequisite, followed by Jenkins installation and configuration.

After installation, Jenkins was accessed through the browser using port 8080 and initial setup was completed.

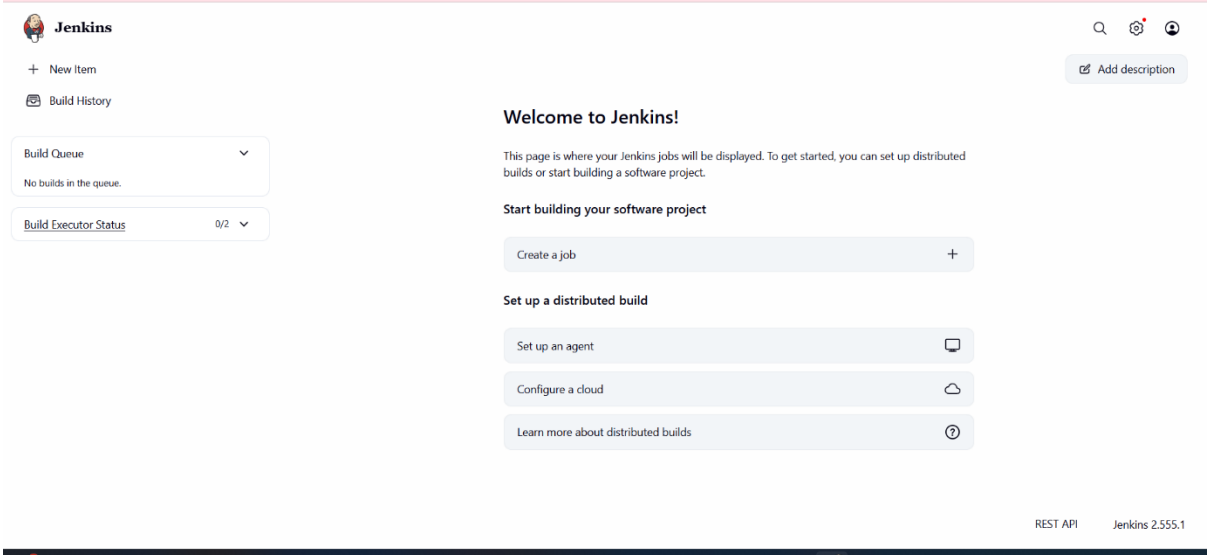
Screenshot 1: Jenkins Plugin Selection



Screenshot 2: Jenkins Setup Complete



Screenshot 3: Jenkins Dashboard Home

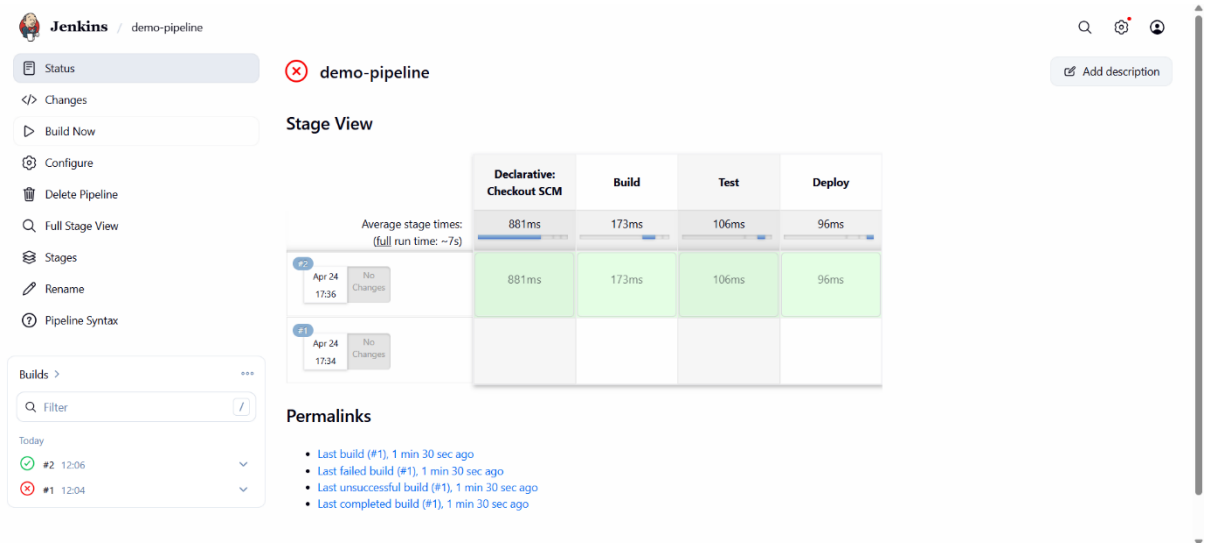


5. CI CD Pipeline Creation

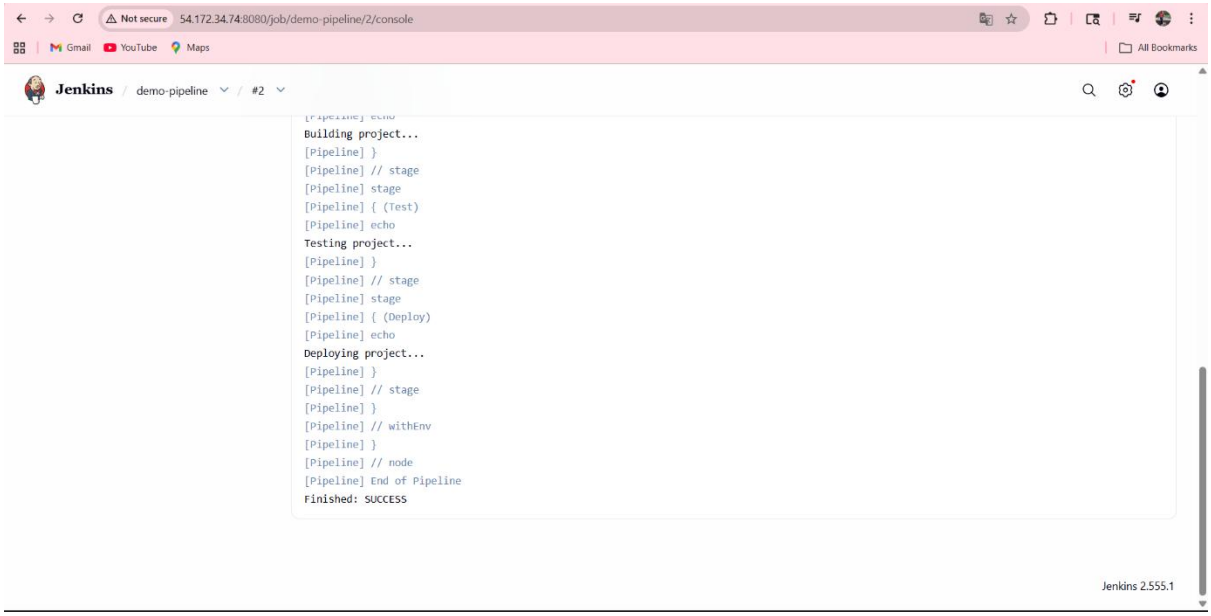
A pipeline was created in Jenkins and connected to a GitHub repository. A Jenkinsfile was added to define pipeline stages such as Build, Test, and Deploy.

The pipeline was executed successfully, demonstrating automated workflow.

Screenshot 4: Pipeline First Execution Success



Screenshot 5: Pipeline Console Output Basic

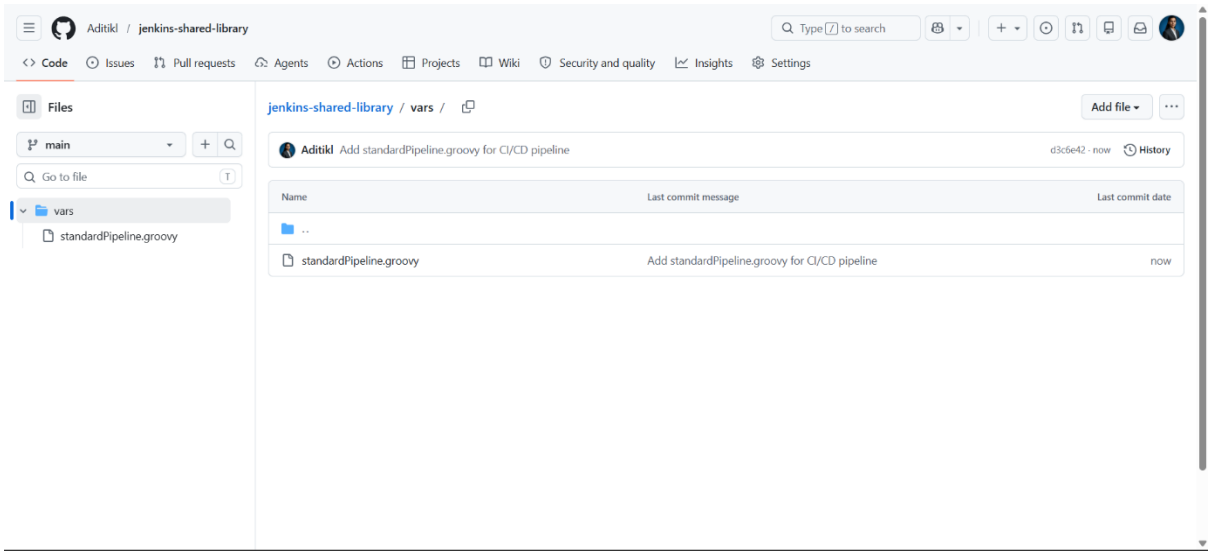


6. Shared Library Implementation

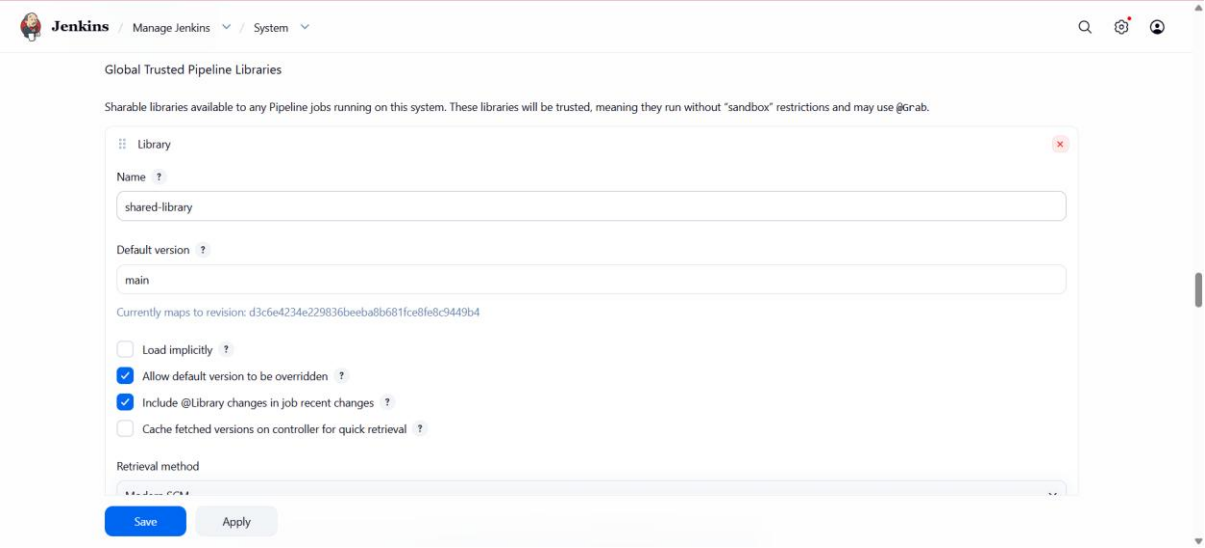
To avoid duplication and standardize pipelines, a Jenkins shared library was created in GitHub. The shared library contains reusable pipeline code stored in a structured format.

The shared library was then configured in Jenkins global settings.

Screenshot 6: Shared Library Repository Structure



Screenshot 7: Jenkins Shared Library Configuration

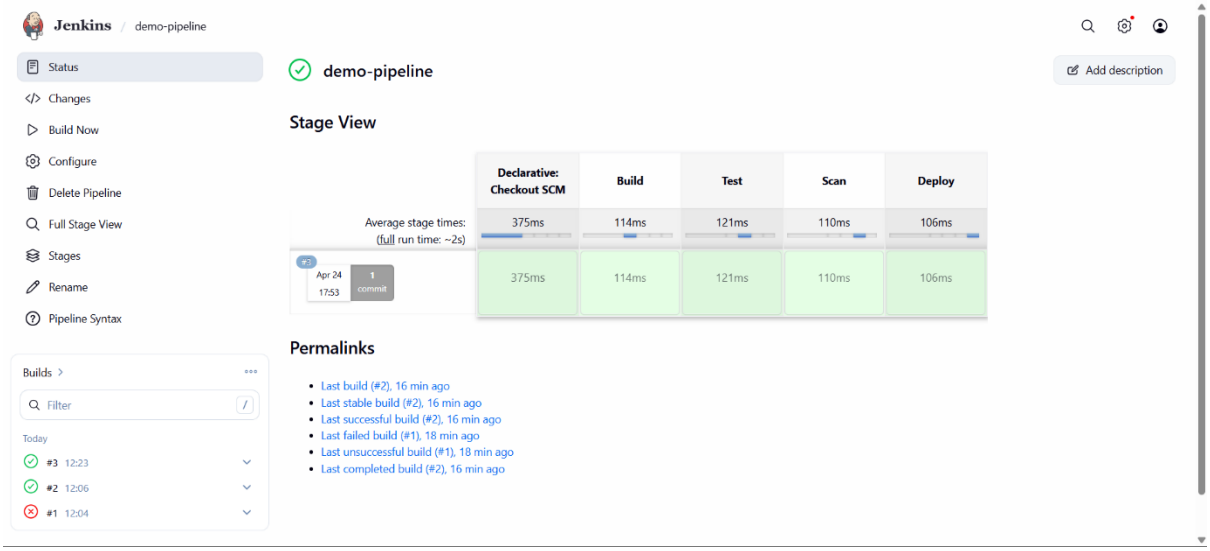


7. Standardized Pipeline Using Shared Library

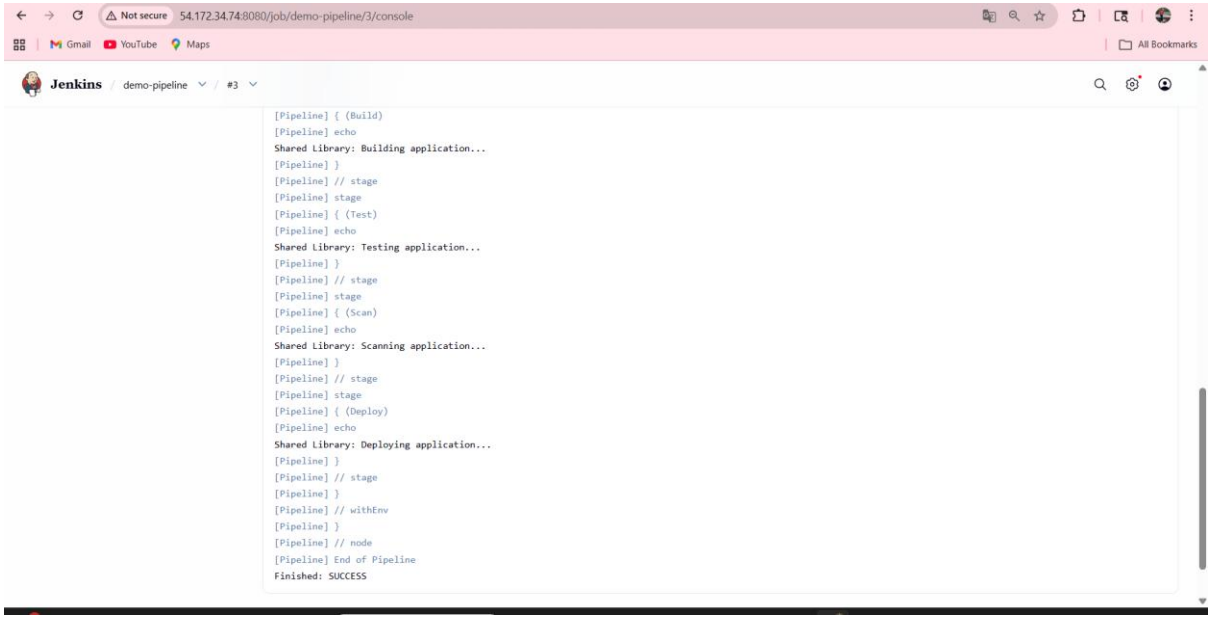
The pipeline was modified to use the shared library instead of writing pipeline code directly. This ensures consistency across applications.

New stages such as Scan were also added through the shared library.

Screenshot 8: Shared Library Pipeline Success



Screenshot 9: Shared Library Console Output



8. Multi-Application Support

A second application was created and connected to Jenkins using the same shared library. This proves that multiple applications can use a centralized CI CD pipeline.

Screenshot 10: Multi Application Pipeline Success

9. Advantages of Centralized CI CD Platform

- Reduces duplication of pipeline code
- Ensures consistency across projects

- Improves maintainability
- Enhances security and control
- Scalable for multiple applications

10. Conclusion

This project successfully demonstrates the implementation of a centralized CI CD platform using Jenkins. By integrating shared libraries and supporting multiple applications, the system ensures efficient, consistent, and scalable software delivery.

11. Future Enhancements

- Integration with Docker for containerized deployment
- Implementation of Kubernetes for orchestration
- Adding security scanning tools
- Automating deployment to cloud environments